



Guide d'accompagnement méthodologique : Jeu de Memory

Conception d'un Jeu de Memory en Vanilla JS — Version Adaptée Public BUT 2A

Étape 1 : Préparation des variables et création du paquet de cartes

1. Liaison avec le HTML (Le DOM)

Pour commencer, créez des variables pour 'attraper' les éléments HTML de votre page (le plateau, les compteurs de coups et de temps, le message de résultat et le bouton rejouer). Cela permettra à votre script JavaScript de modifier l'affichage en temps réel :

```
const board = document.getElementById("game-board");
const movesDisplay = document.getElementById("moves");
const timerDisplay = document.getElementById("timer");
const resultDisplay = document.getElementById("result");
const restartBtn = document.getElementById("restart-btn");
```

2. Les variables de suivi du jeu

Créez ensuite les variables qui vont stocker l'état du jeu à un instant T (si le plateau est verrouillé, le nombre de coups, le temps écoulé, etc.). Pour que les images changent à chaque fois qu'on recharge la page, on utilise un nombre de départ aléatoire (imgStart) :

```
let dimension = 150;
let imgStart = Math.floor(Math.random() * 100) + 1;

let cards = [];
let firstCard = null;
let secondCard = null;
let lockBoard = false;
let moves = 0;
let matchedCount = 0;
let seconds = 0;
let timerInterval = null;
```



3. Fabrication des paires de cartes

Construisez un tableau de 8 images grâce à une boucle comprenant une requête vers **Lorem Picsum** avec pour variables la dimension de l'image et son id aléatoire (imgStart). Ce dernier devrait proposer des images différentes d'une partie à l'autre.

Doublez ces images (nous avons besoin de paires) dans un nouveau tableau grâce au Spread Operator (syntaxe de décomposition en français) :

```
const images = [];  
for (let i = imgStart; i <= imgStart + 7; i++) {  
    images.push(`https://picsum.photos/seed/${i}/${dimension}/${dimension}`);  
}  
cards = [...images, ...images];
```

4. Le mélange des cartes

Pour mélanger le tableau efficacement, on utilise un algorithme classique (**Fisher-Yates**).

On parcourt le tableau en partant de la fin et on échange la carte actuelle avec une autre carte choisie au hasard en utilisant une syntaxe moderne de permutation rapide :

```
function shuffle(array) {  
    for (let i = array.length - 1; i > 0; i--) {  
        const j = Math.floor(Math.random() * (i + 1));  
        [array[i], array[j]] = [array[j], array[i]]; // On échange les deux places  
    }  
}
```



Étape 2 : Lancement d'une partie et création des cartes dans la page (initGame)

La fonction `initGame()` sert à démarrer ou redémarrer une partie. Elle remet tous les compteurs à zéro, efface le plateau visuel, mélange les cartes, puis recrée les éléments dans la page. Pour chaque carte, on ajoute des configurations pour l'accessibilité (pour que le jeu soit jouable au clavier) et on lui attache l'écouteur de clic :

```
function initGame() {
  board.innerHTML = "";
  resultDisplay.textContent = "";
  moves = 0;
  matchedCount = 0;
  seconds = 0;
  firstCard = null;
  secondCard = null;
  lockBoard = false;

  movesDisplay.textContent = `Coups : ${moves}`;
  timerDisplay.textContent = `Temps : 00:00`;

  shuffle(cards);

  cards.forEach((imgUrl) => {
    const card = document.createElement("div");
    card.classList.add("card");
    card.setAttribute("role", "button");
    card.setAttribute("tabindex", "0");
    card.dataset.value = imgUrl; // On cache l'URL de l'image dans la carte
    board.appendChild(card);

    // Quand on clique, on appelle la fonction de gestion du clic
    card.addEventListener("click", () => handleCardClick(card));
  });

  clearInterval(timerInterval);
  startTimer();
}
```



Étape 3 : Gestion du clic, sécurité des actions et révélation

1. Le contrôle du clic et ses 4 barrières de sécurité (handleCardClick)

Dès qu'un joueur clique sur une carte, il faut s'assurer qu'il a le droit de le faire. On met en place une ligne de sécurité avec 4 vérifications logiques interconnectées par l'opérateur OU (||) :

```
if (lockBoard || card.classList.contains("matched") || card === firstCard ||  
card.firstChild) {  
    return; // Si une des conditions est vraie, on ignore le clic et on s'arrête là  
}
```

Analyse détaillée des 4 barrières de sécurité indispensables :

- lockBoard : Bloque le clic si le jeu est en train de patienter pendant l'affichage de deux cartes ratées.
- card.classList.contains('matched') : Empêche d'interagir avec une carte qui a déjà été trouvée et fixée.
- card === firstCard [Protection anti-triche] : Évite que le joueur ne clique deux fois de suite sur la même carte. Sans cela, le jeu verrait que l'image est identique à elle-même et validerait une paire artificielle sur une seule carte !
- card.firstChild [Protection de l'affichage] : Regarde si la carte a déjà un enfant (l'image). Si c'est le cas, elle est déjà face visible et l'action est ignorée.

Code complet de la fonction handleCardClick :

```
function handleCardClick(card) {  
    if (lockBoard || card.classList.contains("matched") || card === firstCard ||  
card.firstChild) {  
        return;  
    }  
  
    revealCard(card); // Tout est bon, on affiche l'image de la carte  
  
    if (!firstCard) {  
        firstCard = card; // C'est la première carte du tour  
        return;  
    }  
  
    secondCard = card;  
    lockBoard = true; // On bloque le plateau le temps de vérifier  
    moves++;  
    movesDisplay.textContent = `Coups : ${moves}`;  
  
    checkMatch();  
}
```



2. L'affichage de la carte (revealCard)

La fonction revealCard s'occupe uniquement de fabriquer la balise HTML , de lui attribuer l'URL et de l'insérer dans la case pour la rendre visible :

```
function revealCard(card) {  
    const img = document.createElement("img");  
    img.src = card.dataset.value;  
    img.alt = "Image de mémoire";  
    card.appendChild(img);  
}
```

Étape 4 : Vérification des paires, Chronomètre et Victoire

1. La comparaison logique (checkMatch)

On compare la valeur cachée (dataset.value) des deux cartes. Si c'est identique, on leur donne la classe '.matched'. Si c'est raté, on attend exactement 800 millisecondes (pour laisser le temps au joueur de mémoriser son erreur) avant de vider le contenu de la carte pour la cacher à nouveau :

```
function checkMatch() {  
    const isMatch = firstCard.dataset.value === secondCard.dataset.value;  
  
    if (isMatch) {  
        firstCard.classList.add("matched");  
        secondCard.classList.add("matched");  
        matchedCount += 2;  
        resetTurn();  
        checkVictory();  
    } else {  
        // On attend 0.8 seconde avant de cacher les images  
        setTimeout(() => {  
            firstCard.innerHTML = "";  
            secondCard.innerHTML = "";  
            resetTurn(); // On débloque le plateau pour le coup suivant  
        }, 800);  
    }  
}  
  
function resetTurn() {  
    firstCard = null;  
    secondCard = null;  
    lockBoard = false;  
}
```



2. Fin de partie et Chronomètre

La victoire se déclenche quand le nombre de cartes validées atteint la taille totale du tableau. Le chronomètre tourne toutes les secondes (1000 ms) et utilise `formatTime` pour afficher proprement les minutes et les secondes avec deux chiffres grâce à `padStart` :

```
function checkVictory() {
  if (matchedCount === cards.length) {
    stopTimer();
    resultDisplay.textContent = `Victoire ! Coups : ${moves} | Temps :
${formatTime(seconds)}`;
  }
}

function startTimer() {
  timerInterval = setInterval(() => {
    seconds++;
    timerDisplay.textContent = `Temps : ${formatTime(seconds)}`;
  }, 1000);
}

function stopTimer() {
  clearInterval(timerInterval);
}

function formatTime(sec) {
  const min = String(Math.floor(sec / 60)).padStart(2, "0");
  const s = String(sec % 60).padStart(2, "0");
  return `${min}:${s}`;
}

// Liaisons finales
restartBtn.addEventListener("click", initGame);
initGame();
```